هندسة البيانات

الدليل الشامل لاحتراف هندسة البيانات

Data Engineering

The Complete Guide to Mastering Data Engineering

د. فضل محمد الاكوع

Dr. Fadhl Mohammed Al-Akwa'a

Arabic & English | عربي - إنجليزي

2026

هندسة البيانات

الدليل الشامل لاحتراف هندسة البيانات

Data Engineering

The Complete Guide to Mastering Data Engineering

د. فضل محمد الأكوع

Dr. Fadhl Mohammed Al-Akwa'a

الطبعة الأولى — 2026

جميع الحقوق محفوظة للمؤلف

بِسۡمِ ٱللَّهِ ٱلرَّحۡمَٰنِ ٱلرَّحِيمِ

إهــداء

إلى كلّ طالب علمٍ يسعى لبناء مستقبلٍ رقميّ أفضل

إلى من يؤمن بأنّ البيانات هي نفط العصر الحديث

إلى كلّ مهندسٍ يبني الجسور بين البيانات الخام والمعرفة

إلى روح والدي رحمه الله

إلى أمّي حفظها الله

إلى زوجتي وأولادي

إلى إخواني وأخواتي

فهرس المحتويات

المقدمة

الفصل الأول: ما هي هندسة البيانات؟

تعريف هندسة البيانات

الفرق بين هندسة البيانات وعلم البيانات

أهمية هندسة البيانات في العصر الرقمي

الفصل الثاني: المهارات الأساسية لمهندس البيانات

لغات البرمجة (Python, SQL, Java, Scala)

قواعد البيانات العلائقية وغير العلائقية

أنظمة التشغيل وسطر الأوامر

الفصل الثالث: قواعد البيانات وأنظمة التخزين

قواعد البيانات العلائقية (MySQL, PostgreSQL)

قواعد البيانات غير العلائقية (MongoDB, Cassandra)

مستودعات البيانات (Data Warehouses)

بحيرات البيانات (Data Lakes)

الفصل الرابع: SQL المتقدم لمهندسي البيانات

الاستعلامات المعقدة والنوافذ التحليلية

تحسين الأداء والفهارس

CTEs والاستعلامات الفرعية المتقدمة

الفصل الخامس: منظومة أباتشي (Apache Ecosystem)

Apache Hadoop و HDFS

Apache Spark

Apache Hive

Apache Kafka

Apache Airflow

الفصل السادس: خطوط أنابيب البيانات (ETL/ELT)

مفهوم ETL و ELT

أدوات ETL الشائعة

تصميم خطوط أنابيب فعالة

الفصل السابع: الحوسبة السحابية لمهندسي البيانات

AWS (Redshift, S3, Glue, EMR)

Google Cloud (BigQuery, Dataflow)

Azure (Synapse, Data Factory)

الفصل الثامن: معالجة البيانات الفورية (Real-time Processing)

البث مقابل الدفعات

Apache Kafka Streams

Apache Flink

الفصل التاسع: جودة البيانات وحوكمتها

أسس جودة البيانات

إدارة البيانات الوصفية (Metadata)

الامتثال والخصوصية (GDPR, CCPA)

الفصل العاشر: المسار الوظيفي لمهندس البيانات

أين يعملون؟

الرواتب حسب الخبرة والمنطقة

المؤهلات والشهادات

المقابلات الوظيفية

الفصل الحادي عشر: مشاريع عملية

بناء خط أنابيب بيانات كامل

بناء مستودع بيانات

الفصل الثاني عشر: مستقبل هندسة البيانات

الذكاء الاصطناعي وهندسة البيانات

التوجهات المستقبلية

الخاتمة

المراجع العلمية

القسم الإنجليزي — English Section

نبذة عن المؤلف

مؤلفات المؤلف

المقدّمة

﴿ وَعَلَّمَ آدَمَ الْأَسْمَاءَ كُلَّهَا ﴾

الحمد لله ربّ العالمين، والصلاة والسلام على أشرف المرسلين سيّدنا محمد وعلى آله وصحبه أجمعين. أمّا بعد:

نعيش اليوم في عصر البيانات، حيث تُنتج البشرية يومياً ما يزيد عن 2.5 كوينتيليون بايت من البيانات. هذا الكمّ الهائل من المعلومات يحتاج إلى من يجمعه وينظّمه ويعالجه ويخزّنه بطريقة تجعله قابلاً للاستخدام والتحليل. هنا يأتي دور مهندس البيانات (Data Engineer)، ذلك المحترف الذي يبني البنية التحتية التي تمكّن المؤسسات من الاستفادة من بياناتها.

يهدف هذا الكتاب إلى تقديم دليل شامل ومتكامل لكلّ من يرغب في دخول مجال هندسة البيانات أو تطوير مهاراته فيه. يغطّي الكتاب كلّ ما يحتاجه القارئ من المفاهيم الأساسية إلى التقنيات المتقدمة، مروراً بالأدوات والمنصات السحابية، وصولاً إلى المسار الوظيفي والرواتب وفرص العمل.

لقد حرصتُ على أن يكون هذا الكتاب ثنائي اللغة (عربي - إنجليزي) لتوسيع دائرة الاستفادة، حيث يبدأ بالقسم العربي الكامل ثمّ يتبعه القسم الإنجليزي. كما حرصتُ على تضمين أمثلة عملية وشروحات مبسّطة تناسب المبتدئين والمتقدّمين على حدّ سواء.

أسأل الله أن يجعل هذا العمل نافعاً لكلّ طالب علم، وأن يكون سبباً في فتح آفاق جديدة أمام شبابنا العربي في هذا المجال الحيوي.

الفصل الأول: ما هي هندسة البيانات؟

تعريف هندسة البيانات

هندسة البيانات (Data Engineering) هي فرع من فروع هندسة البرمجيات يركّز على تصميم وبناء وصيانة الأنظمة والبنية التحتية اللازمة لجمع البيانات ونقلها وتخزينها ومعالجتها. يعمل مهندس البيانات على ضمان تدفّق البيانات بسلاسة من مصادرها المتعدّدة إلى الوجهات المطلوبة، سواء كانت مستودعات بيانات أو أنظمة تحليل أو تطبيقات تعلّم آلي.

يمكن تشبيه مهندس البيانات بالمهندس المدني الذي يبني الطرق والجسور والأنفاق التي تربط المدن ببعضها. فكما أنّ المهندس المدني يصمّم البنية التحتية للنقل، فإنّ مهندس البيانات يصمّم البنية التحتية لنقل البيانات وتخزينها.

الفرق بين هندسة البيانات وعلم البيانات

كثيراً ما يُخلط بين هندسة البيانات (Data Engineering) وعلم البيانات (Data Science). لكنّهما تخصّصان مختلفان ومتكاملان:

المعيار	هندسة البيانات	علم البيانات
التركيز الأساسي	بناء وصيانة البنية التحتية للبيانات	تحليل البيانات واستخراج الأنماط
المهارات الرئيسية	SQL, Python, Spark, ETL, السحابة	الإحصاء، التعلّم الآلي، التصوير البياني
المخرجات	خطوط أنابيب بيانات، مستودعات، APIs	نماذج تنبؤية، تقارير، رؤى تحليلية
الأدوات	Airflow, Kafka, Spark, dbt	Jupyter, TensorFlow, scikit-learn, R
التشبيه	بناء شبكة المياه	تحليل جودة المياه

أهمية هندسة البيانات في العصر الرقمي

تزداد أهمية هندسة البيانات يوماً بعد يوم للأسباب التالية:

1. **الانفجار في حجم البيانات:** تتضاعف البيانات العالمية كلّ سنتين تقريباً. تحتاج المؤسسات إلى بنية تحتية قادرة على التعامل مع هذا الحجم المتنامي.

2. **اتخاذ القرارات المبنية على البيانات:** الشركات الناجحة اليوم هي تلك التي تستطيع تحويل بياناتها إلى قرارات استراتيجية.

3. **الذكاء الاصطناعي والتعلّم الآلي:** لا يمكن بناء نماذج ذكاء اصطناعي فعّالة دون بنية تحتية قوية للبيانات.

4. **الامتثال التنظيمي:** تتطلّب القوانين مثل GDPR وCCPA إدارة دقيقة للبيانات وحمايتها.

5. **الميزة التنافسية:** المؤسسات التي تملك بنية بيانات قوية تتفوّق على منافسيها في سرعة الاستجابة وجودة الخدمات.

الفصل الثاني: المهارات الأساسية لمهندس البيانات

لغات البرمجة

Python

تعدّ Python اللغة الأكثر استخداماً في هندسة البيانات لعدّة أسباب: سهولة التعلّم، غنى المكتبات (Pandas, PySpark, SQLAlchemy)، ودعم واسع من المجتمع. يستخدمها مهندس البيانات في كتابة سكربتات ETL، والتعامل مع APIs، ومعالجة البيانات.

```
# مثال: قراءة ملف CSV ومعالجته باستخدام Pandas
import pandas as pd

df = pd.read_csv('sales_data.csv')
df_clean = df.dropna()
df_grouped = df_clean.groupby('category')
```

```
['revenue'].sum()
df_grouped.to_parquet('sales_summary.parquet')
```

SQL

SQL (لغة الاستعلامات المهيكلة) هي اللغة الأساسية للتعامل مع قواعد البيانات. لا يمكن لأيّ مهندس بيانات أن ينجح دون إتقان SQL بمستوى متقدّم. تُستخدم SQL في الاستعلامات، وبناء الجداول، وتحسين الأداء، وإنشاء العروض (Views).

```
-- مثال: استعلام معقد مع نافذة تحليلية
SELECT
    employee_name,
    department,
    salary,
    AVG(salary) OVER (PARTITION BY department) as
dept_avg,
    RANK() OVER (PARTITION BY department ORDER BY
salary DESC) as salary_rank
FROM employees
WHERE hire_date >= '2020-01-01';
```

Java و Scala

تُستخدم Java وScala بشكل واسع في منظومة أباتشي (Apache Ecosystem). كُتب Apache Spark بلغة Scala، وكُتب Apache Kafka بلغة Java. يحتاج

مهندس البيانات المتقدّم إلى فهم إحدى هاتين اللغتين على الأقلّ للتعامل مع الأنظمة الموزّعة.

Bash / Shell Scripting

مهارة أساسية لأتمتة المهام اليومية مثل نقل الملفات، جدولة المهام (cron jobs)، ومراقبة الأنظمة.

```
#!/bin/bash
# سكربت لنقل ملفات البيانات اليومية
DATE=$(date +%Y%m%d)
SOURCE="/data/raw/${DATE}/"
DEST="s3://data-lake/raw/${DATE}/"

aws s3 sync $SOURCE $DEST --exclude "*.tmp"
echo "Data sync completed for ${DATE}"
```

قواعد البيانات العلائقية وغير العلائقية

يجب على مهندس البيانات فهم الفرق بين أنواع قواعد البيانات واختيار الأنسب لكلّ حالة استخدام:

النوع	أمثلة	الاستخدام الأمثل	نموذج البيانات

علائقية (RDBMS)	PostgreSQL, MySQL	بيانات مهيكلة، معاملات مالية	جداول وعلاقات
مستندات (Document)	MongoDB	بيانات شبه مهيكلة، محتوى متغيّر	مستندات JSON
أعمدة (Columnar)	Cassandra, HBase	بيانات سلاسل زمنية، IoT	عائلات أعمدة
مفتاح-قيمة (Key-Value)	Redis, DynamoDB	تخزين مؤقت، جلسات المستخدمين	أزواج مفتاح-قيمة
رسم بياني (Graph)	Neo4j	شبكات اجتماعية، أنظمة توصيات	عُقد وحواف

أنظمة التشغيل وسطر الأوامر

يعمل معظم مهندسي البيانات على أنظمة Linux. المهارات الأساسية تشمل:

إدارة الملفات: التنقل بين المجلدات، نسخ ونقل الملفات، تغيير الصلاحيات.

معالجة النصوص: أدوات مثل grep, awk, sed لتصفية ومعالجة البيانات النصية.

مراقبة النظام: أوامر مثل top, htop, df, du لمراقبة استهلاك الموارد.

الشبكات: فهم أساسيات SSH, SCP, curl, wget للاتصال بالخوادم البعيدة.

Docker: تعدّ الحاويات (Containers) أساسية في بيئات الإنتاج الحديثة.

```
# أوامر Docker أساسية لمهندسي البيانات
docker pull apache/airflow:2.8.0
docker run -d -p 8080:8080 apache/airflow:2.8.0
docker-compose up -d  # لتشغيل بيئة كاملة
```

الفصل الثالث: قواعد البيانات وأنظمة التخزين

قواعد البيانات العلائقية

PostgreSQL

تعدّ PostgreSQL من أقوى قواعد البيانات العلائقية مفتوحة المصدر. تتميّز بدعمها لأنواع البيانات المتقدّمة (JSON, Array, Geospatial)، والأداء العالي، والامتدادات القوية مثل PostGIS للبيانات الجغرافية وTimescaleDB للسلاسل الزمنية.

```
-- إنشاء جدول مع أنواع بيانات متقدمة في PostgreSQL
CREATE TABLE sensor_data (
    id SERIAL PRIMARY KEY,
    device_id UUID NOT NULL,
    readings JSONB,
    location POINT,
    tags TEXT[],
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- إنشاء فهرس على حقل JSONB
```

```
CREATE INDEX idx_readings ON sensor_data
    USING GIN (readings);

-- استعلام على بيانات JSON
SELECT device_id, readings->>'temperature' as temp
FROM sensor_data
WHERE readings @> '{"status": "active"}';
```

MySQL

MySQL هي قاعدة البيانات الأكثر انتشاراً في تطبيقات الويب. تتميّز بالسرعة في القراءة وسهولة الإعداد. محرّك InnoDB يدعم المعاملات (Transactions) والمفاتيح الخارجية (Foreign Keys).

قواعد البيانات غير العلائقية (NoSQL)

MongoDB

قاعدة بيانات مستندية تخزّن البيانات بصيغة BSON (Binary JSON). مناسبة للتطبيقات التي تتطلّب مرونة في هيكل البيانات وقابلية للتوسّع الأفقي.

```
// إدراج مستند في MongoDB
db.users.insertOne({
    name: "أحمد",
```

```
    email: "ahmad@example.com",
    preferences: {
        language: "ar",
        notifications: true
    },
    tags: ["premium", "active"],
    created_at: new Date()
});

// استعلام مع تجميع (Aggregation)
db.orders.aggregate([
    { $match: { status: "completed" } },
    { $group: { _id: "$category", total: { $sum:
"$amount" } } },
    { $sort: { total: -1 } }
]);
```

Apache Cassandra

قاعدة بيانات عمودية موزّعة مصمّمة للتعامل مع كميات ضخمة من البيانات عبر خوادم متعدّدة. تتميّز بعدم وجود نقطة فشل واحدة (No Single Point of Failure) وقابلية التوسّع الخطّي.

Redis

مخزن بيانات في الذاكرة (In-Memory) فائق السرعة. يُستخدم كقاعدة بيانات وكذاكرة تخزين مؤقت (Cache) ووسيط رسائل (Message Broker).

مستودعات البيانات (Data Warehouses)

مستودع البيانات هو نظام تخزين مركزي مصمّم للتحليلات والتقارير. يجمع البيانات من مصادر متعدّدة في مكان واحد بتنسيق منظّم.

خصائص مستودعات البيانات

مُوجّه نحو الموضوع (Subject-Oriented): البيانات منظّمة حول موضوعات أعمال محدّدة مثل المبيعات والعملاء.

متكامل (Integrated): يجمع بيانات من مصادر متعدّدة بتنسيق موحّد.

غير متغيّر (Non-Volatile): البيانات لا تُحذف أو تُعدّل بعد تحميلها.

مُتغيّر زمنياً (Time-Variant): يحتفظ بالبيانات التاريخية للتحليل عبر الزمن.

أبرز مستودعات البيانات

المنصة	المزوّد	المميزات الرئيسية	نموذج التسعير

Amazon Redshift	AWS	تكامل مع خدمات AWS، أداء عالٍ	حسب الاستخدام أو حجز
Google BigQuery	Google Cloud	بدون خادم، استعلامات بيتابايت	حسب الاستعلام
Snowflake	مستقل	فصل الحوسبة عن التخزين	حسب الاستخدام
Azure Synapse	Microsoft	تكامل مع Power BI	حسب الاستخدام
Databricks	مستقل	Lakehouse، تكامل مع Spark	حسب الاستخدام

بحيرات البيانات (Data Lakes)

بحيرة البيانات هي مستودع تخزين مركزي يحتفظ بالبيانات بصيغتها الخام (Raw) حتى يتمّ الحاجة إليها. على عكس مستودعات البيانات، لا تتطلّب بحيرة البيانات تنظيم البيانات قبل تخزينها.

الفرق بين بحيرة البيانات ومستودع البيانات

المعيار	بحيرة البيانات	مستودع البيانات
نوع البيانات	مهيكلة + شبه مهيكلة + غير مهيكلة	مهيكلة فقط
المعالجة	Schema-on-Read	Schema-on-Write

التكلفة	منخفضة نسبياً	أعلى
المستخدمون	علماء بيانات، مهندسون	محلّلون أعمال
الأداء	متفاوت	محسّن للاستعلامات

مفهوم Lakehouse

ظهر مؤخراً مفهوم Lakehouse الذي يجمع بين مزايا بحيرة البيانات ومستودع البيانات. يوفّر تخزيناً منخفض التكلفة مع دعم للمعاملات (ACID Transactions) وأداء عالٍ في الاستعلامات. أبرز الحلول: Delta Lake من Databricks، وApache Iceberg، وApache Hudi.

الفصل الرابع: SQL المتقدّم لمهندسي البيانات

الاستعلامات المعقّدة والنوافذ التحليلية (Window Functions)

النوافذ التحليلية من أهمّ المهارات التي يحتاجها مهندس البيانات. تسمح بإجراء حسابات على مجموعة من الصفوف المرتبطة بالصف الحالي دون تقليص عدد الصفوف.

```
-- حساب المتوسط المتحرك لآخر 7 أيام
SELECT
    date,
    revenue,
    AVG(revenue) OVER (
        ORDER BY date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) as moving_avg_7d,
    SUM(revenue) OVER (
        PARTITION BY EXTRACT(MONTH FROM date)
        ORDER BY date
    ) as cumulative_monthly
FROM daily_sales;
```

```
-- تحديد الفجوات في السلاسل الزمنية
WITH numbered AS (
    SELECT date,
           ROW_NUMBER() OVER (ORDER BY date) as rn,
           date - INTERVAL '1 day' * ROW_NUMBER() OVER
(ORDER BY date) as grp
    FROM observations
)
SELECT MIN(date) as gap_start, MAX(date) as gap_end
FROM numbered
GROUP BY grp
HAVING COUNT(*) > 1;
```

تحسين الأداء والفهارس

تحسين أداء الاستعلامات مهارة حاسمة. يجب على مهندس البيانات فهم كيفية عمل مخطّط التنفيذ (Execution Plan) واختيار الفهارس المناسبة.

أنواع الفهارس

B-Tree Index: الفهرس الافتراضي، مناسب لعمليات المقارنة (=, >, <, BETWEEN).

Hash Index: مناسب لعمليات المساواة فقط (=)، أسرع من B-Tree لهذا الغرض.

GIN (Generalized Inverted Index): مناسب للبحث في المصفوفات وبيانات JSONB.

GiST (Generalized Search Tree): مناسب للبيانات الجغرافية والنصوص الكاملة.

BRIN (Block Range Index): مناسب للجداول الكبيرة المرتّبة طبيعياً (مثل البيانات الزمنية).

```
-- تحليل خطة التنفيذ
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 12345
AND order_date >= '2024-01-01';

-- إنشاء فهرس مركب (Composite Index)
CREATE INDEX idx_orders_customer_date
ON orders (customer_id, order_date DESC);

-- فهرس جزئي (Partial Index) لتحسين الأداء
CREATE INDEX idx_active_orders
```

```
ON orders (order_date)
WHERE status = 'active';
```

CTEs والاستعلامات الفرعية المتقدّمة

التعبيرات الجدولية المشتركة (Common Table Expressions - CTEs) تجعل الاستعلامات المعقّدة أكثر قراءةً وصيانةً.

```
-- CTE تكرارية لبناء شجرة هرمية
WITH RECURSIVE org_tree AS (
    -- الحالة الأساسية
    SELECT id, name, manager_id, 1 as level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- الحالة التكرارية
    SELECT e.id, e.name, e.manager_id, t.level + 1
    FROM employees e
    JOIN org_tree t ON e.manager_id = t.id
)
SELECT * FROM org_tree ORDER BY level, name;

-- استخدام LATERAL JOIN
SELECT d.department_name, t.top_earner, t.max_salary
```

```
FROM departments d
CROSS JOIN LATERAL (
    SELECT e.name as top_earner, e.salary as max_salary
    FROM employees e
    WHERE e.department_id = d.id
    ORDER BY e.salary DESC
    LIMIT 1
) t;
```

الفصل الخامس: منظومة أباتشي (Apache Ecosystem)

﴿ وَقُلْ رَبِّ زِدْنِي عِلْمًا ﴾

Apache Hadoop

Apache Hadoop هو إطار عمل مفتوح المصدر للمعالجة الموزّعة للبيانات الضخمة. يتكوّن من أربعة مكوّنات رئيسية:

1. **HDFS (Hadoop Distributed File System):** نظام ملفات موزّع يقسّم الملفات الكبيرة إلى كتل (Blocks) بحجم 128MB ويوزّعها على عقد متعدّدة مع نسخ احتياطية (Replication Factor = 3 افتراضياً).

2. **YARN (Yet Another Resource Negotiator):** مدير الموارد الذي يوزّع موارد الحوسبة (CPU والذاكرة) على التطبيقات المختلفة.

3. **MapReduce:** نموذج برمجي للمعالجة الموزّعة يقسّم العمل إلى مرحلتين: Map (التصنيف) وReduce (التجميع).

4. **Hadoop Common:** المكتبات والأدوات المشتركة التي تدعم باقي المكوّنات.

> **ملاحظة:** رغم أنّ MapReduce كان ثورياً في وقته، إلا أنّ Apache Spark قد حلّ محلّه في معظم حالات الاستخدام بفضل أدائه الأسرع بمئة ضعف بسبب المعالجة في الذاكرة (In-Memory Processing).

Apache Spark

Apache Spark هو محرّك حوسبة موزّعة موحّد يدعم معالجة الدفعات (Batch) والبثّ المباشر (Streaming) والتعلّم الآلي ومعالجة الرسوم البيانية. يعدّ الأداة الأكثر أهمية في ترسانة مهندس البيانات الحديث.

مكوّنات Spark

Spark Core: المحرّك الأساسي للمعالجة الموزّعة.

Spark SQL: وحدة للعمل مع البيانات المهيكلة باستخدام SQL و DataFrames.

Spark Streaming / Structured Streaming: للمعالجة الفورية.

MLlib: مكتبة التعلّم الآلي الموزّع.

GraphX: لمعالجة الرسوم البيانية.

```
# مثال PySpark: معالجة بيانات المبيعات
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, avg, window

spark =
SparkSession.builder     .appName("SalesAnalysis")
.config("spark.sql.adaptive.enabled",
"true")     .getOrCreate()

# قراءة بيانات من Parquet
df = spark.read.parquet("s3://data-lake/sales/")

# تحويلات وتحليل
result = df     .filter(col("year") ==
2025)     .groupBy("region",
"product_category")     .agg(
        sum("revenue").alias("total_revenue"),
        avg("quantity").alias("avg_quantity"),
        count("order_id").alias("order_count")
    )     .orderBy(col("total_revenue").desc())

# حفظ النتائج
result.write     .mode("overwrite")     .partitionBy("r
egion")     .parquet("s3://data-warehouse/
sales_summary/")
```

Apache Hive

Apache Hive هو نظام مستودع بيانات مبني على Hadoop يوفّر واجهة SQL (تسمى HiveQL) للاستعلام عن البيانات المخزّنة في HDFS. كان Hive ثورياً لأنّه سمح لمحلّلي البيانات الذين يعرفون SQL بالعمل مع البيانات الضخمة دون كتابة كود MapReduce.

خصائص Hive الرئيسية

Schema on Read: لا يفرض هيكلاً على البيانات عند الكتابة.

Metastore: مستودع مركزي للبيانات الوصفية (أسماء الجداول، الأعمدة، الأنواع).

Partitioning: تقسيم الجداول لتحسين أداء الاستعلامات.

Bucketing: توزيع البيانات داخل الأقسام لتحسين عمليات الربط (Join).

```
-- إنشاء جدول Hive مع التقسيم
CREATE EXTERNAL TABLE sales (
    order_id STRING,
    customer_id INT,
    product STRING,
    amount DECIMAL(10,2),
    order_time TIMESTAMP
```

```
)
PARTITIONED BY (year INT, month INT)
STORED AS PARQUET
LOCATION '/data/warehouse/sales/';

-- استعلام مع تحسين الأداء
SET hive.exec.dynamic.partition.mode=nonstrict;

SELECT product, SUM(amount) as total_sales
FROM sales
WHERE year = 2025 AND month BETWEEN 1 AND 6
GROUP BY product
ORDER BY total_sales DESC
LIMIT 20;
```

Apache Kafka

Apache Kafka هو منصة بثّ أحداث موزّعة (Distributed Event Streaming Platform) تُستخدم لبناء خطوط أنابيب البيانات الفورية. يمكنها التعامل مع ملايين الرسائل في الثانية.

المفاهيم الأساسية في Kafka

Topic (الموضوع): قناة تدفّق البيانات التي ينشر فيها المنتجون ويقرأ منها المستهلكون.

Partition (القسم): تقسيم الموضوع لتوزيع الحمل وتحقيق التوازي.

Producer (المنتج): التطبيق الذي ينشر الرسائل.

Consumer (المستهلك): التطبيق الذي يقرأ الرسائل.

Consumer Group: مجموعة مستهلكين تعمل معاً لقراءة موضوع.

Broker (الوسيط): خادم Kafka الذي يخزّن ويخدم البيانات.

```
# Python Kafka Producer
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers=['kafka1:9092', 'kafka2:9092'],
    value_serializer=lambda v:
json.dumps(v).encode('utf-8')
)

# إرسال حدث
event = {
    "user_id": 12345,
    "action": "purchase",
    "product_id": "SKU-789",
    "timestamp": "2025-03-15T10:30:00Z"
}
```

```
producer.send('user-events', value=event)
producer.flush()
```

Apache Airflow

Apache Airflow هو منصة لإدارة وجدولة ومراقبة خطوط العمل (Workflows). يستخدم مفهوم DAG (Directed Acyclic Graph) لتعريف تسلسل المهام وعلاقاتها.

```
# مثال DAG في Airflow
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.operators.s3 import
S3CopyObjectOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data-team',
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
    'email_on_failure': True,
}

with DAG(
    'daily_etl_pipeline',
```

```
    default_args=default_args,
    schedule_interval='@daily',
    start_date=datetime(2025, 1, 1),
    catchup=False,
) as dag:

    extract = PythonOperator(
        task_id='extract_data',
        python_callable=extract_from_api,
    )

    transform = PythonOperator(
        task_id='transform_data',
        python_callable=clean_and_transform,
    )

    load = PythonOperator(
        task_id='load_to_warehouse',
        python_callable=load_to_redshift,
    )

    extract >> transform >> load
```

الفصل السادس: خطوط أنابيب البيانات (ETL/ELT)

مفهوم ETL و ELT

ETL (Extract, Transform, Load) وELT (Extract, Load, Transform) هما نمطان أساسيان لنقل البيانات من المصادر إلى الوجهات.

ETL التقليدي

Extract (الاستخراج): جمع البيانات من المصادر المختلفة (قواعد بيانات، APIs، ملفات).

Transform (التحويل): تنظيف وتحويل البيانات في خادم وسيط قبل تحميلها.

Load (التحميل): تحميل البيانات المحوّلة إلى المستودع النهائي.

ELT الحديث

في نمط ELT، تُحمّل البيانات الخام أولاً إلى المستودع (مثل BigQuery أو Snowflake) ثمّ تُجرى التحويلات داخل المستودع نفسه باستخدام SQL. هذا النمط أصبح أكثر شيوعاً بفضل قدرة المستودعات السحابية الحديثة على المعالجة.

المعيار	ETL	ELT
مكان التحويل	خادم وسيط	داخل المستودع
الأنسب لـ	بيانات حساسة، تحويلات معقدة	بيانات ضخمة، مستودعات سحابية
السرعة	أبطأ (نقل مزدوج)	أسرع (نقل واحد)
المرونة	أقل مرونة	مرن جداً
الأدوات	Informatica, Talend	dbt, Snowflake, BigQuery

أدوات ETL/ELT الشائعة

dbt (Data Build Tool)

أداة تحويل حديثة تعمل داخل مستودع البيانات. تتيح لمهندسي البيانات كتابة التحويلات بلغة SQL مع دعم للاختبارات والتوثيق وإدارة الإصدارات.

```sql
-- نموذج dbt لتحويل بيانات العملاء
-- models/marts/customers.sql

WITH source AS (
    SELECT * FROM {{ ref('stg_customers') }}
),

orders AS (
    SELECT * FROM {{ ref('stg_orders') }}
),

customer_orders AS (
    SELECT
        c.customer_id,
        c.customer_name,
        COUNT(o.order_id) as total_orders,
        SUM(o.amount) as lifetime_value,
        MIN(o.order_date) as first_order_date,
        MAX(o.order_date) as last_order_date
    FROM source c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.customer_name
)

SELECT * FROM customer_orders
```

Apache NiFi

منصة تكامل بيانات مرئية تتيح تصميم خطوط أنابيب بطريقة السحب والإفلات (Drag-and-Drop). مناسبة للفرق التي تفضّل الواجهة المرئية.

Fivetran و Airbyte

أدوات استخراج وتحميل (EL) تتصل بمئات المصادر تلقائياً. Fivetran منتج تجاري، بينما Airbyte مفتوح المصدر.

تصميم خطوط أنابيب فعّالة

مبادئ تصميم خطوط أنابيب البيانات الناجحة:

1. **الاستقرارية (Idempotency):** تشغيل نفس العملية عدّة مرات يُنتج نفس النتيجة.

2. **المراقبة (Monitoring):** تتبّع حالة كلّ مرحلة والتنبيه عند الأخطاء.

3. **قابلية التوسّع (Scalability):** القدرة على التعامل مع أحجام بيانات متزايدة.

4. **التوثيق:** توثيق كلّ مرحلة ومصدر بيانات وقاعدة تحويل.

5. **معالجة الأخطاء:** آليات إعادة المحاولة وقوائم الأخطاء (Dead Letter Queues).

6. **اختبار البيانات:** التحقّق من جودة البيانات في كلّ مرحلة.

الفصل السابع: الحوسبة السحابية لمهندسي البيانات

Amazon Web Services (AWS)

AWS هي المنصة السحابية الأكثر استخداماً وتوفّر مجموعة شاملة من خدمات البيانات:

Amazon S3 (Simple Storage Service)

خدمة تخزين كائنات (Object Storage) غير محدودة السعة. تُستخدم كبحيرة بيانات وكمستودع للبيانات الخام والمعالجة. تدعم عدّة فئات تخزين حسب تكرار الوصول.

Amazon Redshift

مستودع بيانات سحابي بيتابايت المستوى. يستخدم معالجة عمودية متوازية (MPP) لاستعلامات فائقة السرعة.

AWS Glue

خدمة ETL بدون خادم تكتشف البيانات وتحوّلها تلقائياً. تتضمّن Glue Catalog لإدارة البيانات الوصفية.

Amazon EMR (Elastic MapReduce)

منصة سحابية لتشغيل أطر العمل مثل Spark وHadoop وHive وPrestoDB.

Amazon Kinesis

خدمة بثّ بيانات فوري لجمع ومعالجة وتحليل البيانات المتدفّقة في الوقت الحقيقي.

Google Cloud Platform (GCP)

Google BigQuery

مستودع بيانات بدون خادم (Serverless) يمكنه معالجة بيتابايت من البيانات في ثوانٍ. يتميّز بنموذج تسعير مرن (حسب الاستعلام أو حسب السعة).

```
-- استعلام BigQuery على بيانات ضخمة
SELECT
    EXTRACT(DATE FROM timestamp) as date,
    country,
```

```
    COUNT(DISTINCT user_id) as unique_users,
    SUM(revenue) as total_revenue,
    APPROX_QUANTILES(session_duration, 100)[OFFSET(50)]
as median_duration
FROM `project.dataset.events`
WHERE DATE(timestamp) BETWEEN '2025-01-01' AND
'2025-06-30'
GROUP BY 1, 2
HAVING unique_users > 1000
ORDER BY total_revenue DESC;
```

Google Cloud Dataflow

خدمة معالجة بيانات موحّدة للدفعات والبثّ المباشر. مبنية على Apache Beam.

Google Cloud Pub/Sub

خدمة رسائل فورية لبناء أنظمة مبنية على الأحداث (Event-Driven).

Microsoft Azure

Azure Synapse Analytics

خدمة تحليلات شاملة تجمع بين مستودعات البيانات ومعالجة البيانات الضخمة والتكامل مع Power BI.

Azure Data Factory

خدمة ETL/ELT سحابية مع واجهة مرئية لتصميم خطوط أنابيب البيانات.

Azure Databricks

منصة تحليلات مبنية على Apache Spark، مُحسّنة للعمل على Azure.

مقارنة بين المنصات السحابية الثلاث

الخدمة	AWS	GCP	Azure
مستودع بيانات	Redshift	BigQuery	Synapse
تخزين كائنات	S3	Cloud Storage	Blob Storage
ETL	Glue	Dataflow	Data Factory
بثّ مباشر	Kinesis	Pub/Sub	Event Hubs
Spark	EMR	Dataproc	Databricks/ HDInsight
جدولة	Step Functions	Cloud Composer	Data Factory

الفصل الثامن: معالجة البيانات الفورية (Real-time Processing)

البثّ مقابل الدفعات

هناك نمطان أساسيان لمعالجة البيانات:

معالجة الدفعات (Batch Processing): معالجة كميات كبيرة من البيانات المجمّعة في فترات محدّدة (كلّ ساعة، يومياً). مناسبة للتقارير والتحليلات التاريخية.

معالجة البثّ (Stream Processing): معالجة البيانات فور وصولها بتأخير ضئيل (ملّي ثانية إلى ثوانٍ). مناسبة للتنبيهات الفورية وكشف الاحتيال والتوصيات اللحظية.

معمارية Lambda

تجمع بين طبقة الدفعات وطبقة البثّ لتقديم رؤية شاملة. رغم فعاليتها، تعاني من تعقيد الصيانة بسبب ازدواجية المنطق.

معمارية Kappa

تعتمد على طبقة البثّ فقط، حيث تُعالج جميع البيانات كتيار مستمر. أبسط في التصميم ولكنّها تتطلّب أنظمة بثّ قوية.

Apache Kafka Streams

مكتبة جافا لمعالجة تيارات البيانات مباشرة من Kafka. تتميّز بالبساطة وعدم الحاجة لبنية تحتية إضافية.

Apache Flink

محرّك معالجة تيارات بيانات قوي يدعم المعالجة الفورية والدفعية. يتميّز بدعم ممتاز لإدارة الحالة (Stateful Processing) والنوافذ الزمنية (Windowing).

```
# مثال: معالجة فورية باستخدام Spark Structured
Streaming
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark =
SparkSession.builder.appName("RealTimeAnalytics").getOr
Create()
```

```
# قراءة من Kafka
df =
spark.readStream     .format("kafka")     .option("kafk
a.bootstrap.servers", "kafka:
9092")     .option("subscribe", "user-
events")     .load()

# تحويل وتحليل
events = df.select(
    from_json(col("value").cast("string"),
schema).alias("data")
).select("data.*")

# تجميع بنافذة زمنية
windowed = events     .withWatermark("timestamp", "10
minutes")     .groupBy(
        window("timestamp", "5 minutes"),
        "event_type"
    )     .count()

# كتابة النتائج
query =
windowed.writeStream     .outputMode("update")     .for
mat("console")     .start()

query.awaitTermination()
```

الفصل التاسع: جودة البيانات وحوكمتها

أسس جودة البيانات

جودة البيانات هي العمود الفقري لأيّ نظام بيانات ناجح. البيانات الرديئة تؤدّي إلى قرارات خاطئة وتكاليف باهظة. أبعاد جودة البيانات الستة:

1. **الدقّة (Accuracy)**: مدى تطابق البيانات مع الواقع.
2. **الاكتمال (Completeness)**: عدم وجود قيم مفقودة أو ناقصة.
3. **الاتساق (Consistency)**: عدم وجود تناقضات بين مصادر البيانات المختلفة.
4. **الحداثة (Timeliness)**: توفّر البيانات في الوقت المناسب.
5. **الصلاحية (Validity)**: التزام البيانات بالقواعد والقيود المحدّدة.
6. **التفرّد (Uniqueness)**: عدم وجود تكرارات غير مبرّرة.

أدوات جودة البيانات

Great Expectations: إطار عمل Python لاختبار جودة البيانات. يتيح تعريف "توقّعات" (Expectations) والتحقّق منها تلقائياً.

dbt Tests: اختبارات مدمجة في dbt للتحقّق من القيم الفريدة والعلاقات والقيم المقبولة.

Soda: منصة مراقبة جودة البيانات تدعم SQL وSpark.

```
# مثال Great Expectations
import great_expectations as gx

context = gx.get_context()
datasource = context.sources.add_pandas("my_source")
data_asset =
datasource.add_dataframe_asset("sales_data")

# تعريف التوقعات
batch = data_asset.get_batch()
batch.expect_column_values_to_not_be_null("customer_id"
)
batch.expect_column_values_to_be_between("quantity", 1,
10000)
batch.expect_column_values_to_be_in_set("status",
    ["pending", "shipped", "delivered", "cancelled"])
```

```
# التحقق
results = batch.validate()
print(f"Success: {results.success}")
```

إدارة البيانات الوصفية (Metadata Management)

البيانات الوصفية هي "البيانات عن البيانات". تتضمّن معلومات عن الهيكل والمصدر والجودة والعلاقات. أدوات إدارة البيانات الوصفية تشمل:

Apache Atlas: منصة حوكمة بيانات مفتوحة المصدر.

DataHub: منصة اكتشاف بيانات من LinkedIn.

Amundsen: أداة اكتشاف بيانات من Lyft.

dbt Docs: توثيق تلقائي لنماذج البيانات.

كتالوج البيانات (Data Catalog)

كتالوج البيانات هو فهرس مركزي لجميع أصول البيانات في المؤسسة. يتيح للمستخدمين اكتشاف البيانات وفهمها والثقة بها.

الامتثال والخصوصية

GDPR (اللائحة العامة لحماية البيانات)

لائحة الاتحاد الأوروبي التي تنظّم جمع ومعالجة البيانات الشخصية. تتطلّب من المؤسسات:

الحصول على موافقة صريحة لجمع البيانات، تمكين حقّ الحذف (Right to be Forgotten)، الإبلاغ عن الاختراقات خلال 72 ساعة، وتعيين مسؤول حماية بيانات (DPO).

CCPA (قانون خصوصية المستهلك في كاليفورنيا)

قانون مشابه لـ GDPR يحمي خصوصية المستهلكين في كاليفورنيا.

تقنيات حماية البيانات

التشفير (Encryption): تشفير البيانات أثناء النقل وأثناء التخزين.

إخفاء الهوية (Anonymization): إزالة أو تعديل المعلومات الشخصية.

الإخفاء الجزئي (Masking): إخفاء أجزاء من البيانات الحساسة مع الحفاظ على بنيتها.

التحكّم في الوصول (RBAC): تحديد صلاحيات الوصول حسب الأدوار.

الفصل العاشر: المسار الوظيفي لمهندس البيانات

﴿ وَأَن لَّيْسَ لِلْإِنسَانِ إِلَّا مَا سَعَىٰ ﴾

أين يعمل مهندسو البيانات؟

يعمل مهندسو البيانات في مختلف القطاعات والصناعات:

شركات التقنية الكبرى: Google, Amazon, Meta, Microsoft, Apple — تمتلك فرق هندسة بيانات ضخمة تعالج بيتابايت يومياً.

القطاع المالي: البنوك وشركات التأمين وصناديق الاستثمار — تعتمد على البيانات في إدارة المخاطر وكشف الاحتيال.

التجارة الإلكترونية: Amazon, Alibaba, Shopify — تستخدم البيانات في التوصيات والتسعير الديناميكي.

الرعاية الصحية: المستشفيات وشركات الأدوية — تحليل السجلات الطبية والأبحاث السريرية.

النقل والخدمات اللوجستية: Uber, Lyft, FedEx — تحسين المسارات والعمليات.

شركات الاستشارات: Deloitte, McKinsey, Accenture — تقديم حلول بيانات للعملاء.

الشركات الناشئة: كثير من الشركات الناشئة تبني فرق بيانات من البداية.

العمل الحرّ (Freelancing): منصات مثل Toptal, Upwork, Fiverr.

الرواتب حسب الخبرة والمنطقة

الولايات المتحدة الأمريكية

المستوى	الراتب السنوي (USD)	الخبرة
مبتدئ (Junior)	$80,000 - $110,000	0-2 سنوات
متوسط (Mid-level)	$110,000 - $150,000	2-5 سنوات
أقدم (Senior)	$150,000 - $200,000	5-8 سنوات
رئيسي (Staff/Principal)	$200,000 - $300,000+	8+ سنوات

مدير (Manager)	$180,000 - $250,000	+6 سنوات

أوروبا

المستوى	الراتب السنوي (EUR)	الملاحظات
مبتدئ	€40,000 - €60,000	المملكة المتحدة وألمانيا أعلى
متوسط	€60,000 - €90,000	هولندا وسويسرا أعلى
أقدم	€90,000 - €130,000	لندن الأعلى في أوروبا

الشرق الأوسط

المستوى	الراتب السنوي (USD)	الملاحظات
مبتدئ	$40,000 - $65,000	الإمارات والسعودية الأعلى
متوسط	$65,000 - $100,000	معفى من الضرائب غالباً
أقدم	$100,000 - $160,000	مع بدلات سكن ونقل

نصيحة: الرواتب عن بُعد (Remote) من شركات أمريكية قد تصل إلى 80-90% من الراتب المحلي، مما يجعلها خياراً ممتازاً للمهندسين في المنطقة العربية.

المؤهّلات والشهادات

المسار الأكاديمي

الدرجة الجامعية: علوم الحاسب، هندسة البرمجيات، نظم المعلومات، الرياضيات، أو الإحصاء. ليست إلزامية لكنّها مفضّلة.

الماجستير: ماجستير في علوم البيانات أو هندسة البرمجيات يمنح ميزة تنافسية.

التعلّم الذاتي: كثير من مهندسي البيانات الناجحين تعلّموا ذاتياً من منصات مثل Coursera, Udemy, DataCamp.

الشهادات المهنية المعتمدة

الشهادة	المزوّد	المستوى	التكلفة التقريبية
AWS Data Analytics Specialty	Amazon	متقدم	$300
Google Professional Data Engineer	Google	متقدم	$200
Azure Data Engineer Associate	Microsoft	متوسط	$165
Databricks Data Engineer Associate	Databricks	متوسط	$200
Snowflake SnowPro Core	Snowflake	أساسي	$175
Apache Spark Developer	Databricks	متقدم	$200

المقابلات الوظيفية

تتضمّن مقابلات مهندسي البيانات عادةً:

1. **أسئلة SQL:** استعلامات معقّدة، نوافذ تحليلية، تحسين أداء.
2. **تصميم الأنظمة:** تصميم خطّ أنابيب بيانات كامل لحالة استخدام معيّنة.
3. **البرمجة:** Python/Java مع التركيز على معالجة البيانات.
4. **المفاهيم:** الفرق بين أنماط التخزين، مبادئ الحوسبة الموزّعة.
5. **الخبرة العملية:** مناقشة مشاريع سابقة والتحدّيات التي واجهتها.

> **نصيحة للمقابلات:** ركّز على فهم المبادئ الأساسية (CAP Theorem, ACID, BASE) أكثر من حفظ أوامر الأدوات. الأدوات تتغيّر، لكنّ المبادئ تبقى.

الفصل الحادي عشر: مشاريع عملية

مشروع 1: بناء خطّ أنابيب بيانات كامل

في هذا المشروع سنبني خطّ أنابيب بيانات (Data Pipeline) من البداية إلى النهاية:

وصف المشروع

بناء نظام يجمع بيانات الطقس من API خارجي، يعالجها، ويخزّنها في مستودع بيانات للتحليل.

المكوّنات

المصدر: OpenWeatherMap API

الاستخراج: Python script مجدول بـ Airflow

التخزين الخام: S3 بصيغة JSON

التحويل: Spark أو dbt

المستودع: PostgreSQL أو Redshift

التصوير: Grafana أو Metabase

```
# خطوات بناء المشروع
# 1. إعداد البيئة
mkdir weather-pipeline && cd weather-pipeline
python -m venv venv
pip install apache-airflow pyspark boto3 requests

# 2. هيكل المشروع
# weather-pipeline/
# ├── dags/
# │   └── weather_etl.py
# ├── src/
# │   ├── extract.py
# │   ├── transform.py
# │   └── load.py
# ├── tests/
# │   └── test_transform.py
# ├── docker-compose.yml
# └── requirements.txt
```

```
# src/extract.py - استخراج بيانات الطقس
import requests
import json
from datetime import datetime

def extract_weather(city, api_key):
```

```
    # استخراج بيانات الطقس من OpenWeatherMap
    url = f"http://api.openweathermap.org/data/2.5/
weather"
    params = {
        "q": city,
        "appid": api_key,
        "units": "metric"
    }

    response = requests.get(url, params=params)
    response.raise_for_status()

    data = response.json()
    data['extracted_at'] =
datetime.utcnow().isoformat()

    return data

# src/transform.py - تحويل البيانات
def transform_weather(raw_data):
    # تنظيف وتحويل بيانات الطقس
    return {
        "city": raw_data["name"],
        "country": raw_data["sys"]["country"],
        "temperature": raw_data["main"]["temp"],
        "humidity": raw_data["main"]["humidity"],
        "pressure": raw_data["main"]["pressure"],
        "wind_speed": raw_data["wind"]["speed"],
```

```
        "description": raw_data["weather"][0]
["description"],
        "extracted_at": raw_data["extracted_at"]
    }
```

مشروع 2: بناء مستودع بيانات

بناء مستودع بيانات باستخدام نموذج النجمة (Star Schema) لتحليل بيانات مبيعات:

```
-- إنشاء جداول البُعد (Dimension Tables)
CREATE TABLE dim_date (
    date_key INT PRIMARY KEY,
    full_date DATE,
    day_of_week VARCHAR(10),
    month_name VARCHAR(10),
    quarter INT,
    year INT,
    is_weekend BOOLEAN
);

CREATE TABLE dim_product (
    product_key INT PRIMARY KEY,
    product_id VARCHAR(50),
    product_name VARCHAR(200),
    category VARCHAR(100),
```

```
    subcategory VARCHAR(100),
    brand VARCHAR(100),
    unit_price DECIMAL(10,2)
);

CREATE TABLE dim_customer (
    customer_key INT PRIMARY KEY,
    customer_id VARCHAR(50),
    customer_name VARCHAR(200),
    email VARCHAR(200),
    city VARCHAR(100),
    country VARCHAR(100),
    segment VARCHAR(50)
);

-- جدول الحقائق (Fact Table)
CREATE TABLE fact_sales (
    sale_key BIGINT PRIMARY KEY,
    date_key INT REFERENCES dim_date,
    product_key INT REFERENCES dim_product,
    customer_key INT REFERENCES dim_customer,
    quantity INT,
    unit_price DECIMAL(10,2),
    discount DECIMAL(5,2),
    total_amount DECIMAL(12,2),
    profit DECIMAL(12,2)
);
```

الفصل الثاني عشر: مستقبل هندسة البيانات

الذكاء الاصطناعي وهندسة البيانات

يُحدث الذكاء الاصطناعي تحوّلاً جذرياً في مجال هندسة البيانات:

أتمتة خطوط الأنابيب: أدوات مثل Auto-ETL تستخدم الذكاء الاصطناعي لاكتشاف المصادر وبناء خطوط الأنابيب تلقائياً.

جودة البيانات الذكية: نماذج ML تكتشف الشذوذ (Anomalies) والأنماط غير الطبيعية في البيانات تلقائياً.

الاستعلام باللّغة الطبيعية: واجهات تتيح الاستعلام عن البيانات باللّغة الطبيعية بدلاً من SQL.

MLOps: دمج عمليات التعلّم الآلي مع هندسة البيانات.

التوجّهات المستقبلية

1. Data Mesh

نهج لامركزي لإدارة البيانات حيث تمتلك كلّ فرقة أعمال (Domain) بياناتها وتقدّمها كمنتج.

2. Real-time Everything

الانتقال من المعالجة الدفعية إلى المعالجة الفورية كمعيار افتراضي.

3. Data Contracts

اتفاقيات رسمية بين منتجي البيانات ومستهلكيها تحدّد الهيكل والجودة والتوفّر.

4. Declarative Data Pipelines

تعريف "ماذا" نريد بدلاً من "كيف" نحقّقه، مع ترك التنفيذ للمحرّكات الذكية.

5. Lakehouse Architecture

استمرار تبنّي معمارية Lakehouse التي تجمع مزايا بحيرات البيانات ومستودعاتها.

6. الحوسبة الحافّية (Edge Computing)

معالجة البيانات قريباً من مصدرها (أجهزة IoT، الهواتف) بدلاً من إرسالها إلى السحابة.

الخاتمة

في ختام هذا الكتاب، نأمل أن نكون قد قدّمنا للقارئ العربي دليلاً شاملاً ومتكاملاً في مجال هندسة البيانات. لقد تناولنا الموضوع من جوانبه المتعدّدة، بدءاً من المفاهيم الأساسية وصولاً إلى التقنيات المتقدّمة والتوجّهات المستقبلية.

هندسة البيانات ليست مجرّد مجموعة أدوات وتقنيات، بل هي فلسفة في التعامل مع البيانات وتحويلها إلى أصول قيّمة. المهندس الناجح هو من يفهم المبادئ الأساسية ويستطيع تطبيقها بغضّ النظر عن الأداة المستخدمة.

ننصح القارئ بالبدء بالمشاريع العملية فور الانتهاء من قراءة هذا الكتاب. التعلّم الحقيقي يأتي من التطبيق العملي ومواجهة التحدّيات الحقيقية. ابدأ بمشروع صغير واجعله ينمو مع نموّ مهاراتك.

أسأل الله أن يوفّق كلّ طالب علم وأن يجعل هذا العمل في ميزان حسناتنا.

والله أعلم

المراجع العلمية

مراجع هندسة البيانات والحوسبة

Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering*. O'Reilly Media.

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley.

Narkhede, N., Shapira, G., & Palino, T. (2017). *Kafka: The Definitive Guide*. O'Reilly Media.

Chambers, B., & Zaharia, M. (2018). *Spark: The Definitive Guide*. O'Reilly Media.

Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit*. 3rd ed. Wiley.

Inmon, W. H. (2005). *Building the Data Warehouse*. 4th ed. Wiley.

White, T. (2015). *Hadoop: The Definitive Guide*. 4th ed. O'Reilly Media.

Dehghani, Z. (2022). *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media.

مراجع SQL وقواعد البيانات

Molinaro, A., & Graaf, R. (2020). *SQL Cookbook*. 2nd ed. O'Reilly Media.

Petrov, A. (2019). *Database Internals*. O'Reilly Media.

Schwartz, B., Zaitsev, P., & Tkachenko, V. (2012). *High Performance MySQL*. O'Reilly Media.

مراجع الحوسبة السحابية

Lisdorf, A. (2021). *Cloud Computing Basics: A Non-Technical Introduction*. Apress.

Amazon Web Services. (2025). *AWS Well-Architected Framework — Data Analytics Lens*.

Google Cloud. (2025). *Google Cloud Architecture Framework*.

مراجع التفسير القرآني

ابن كثير، إسماعيل بن عمر. تفسير القرآن العظيم. دار الفكر.

الطبري، محمد بن جرير. جامع البيان في تأويل القرآن. مؤسسة الرسالة.

القرطبي، محمد بن أحمد. الجامع لأحكام القرآن. دار الكتب المصرية.

مصادر إلكترونية

//:Apache Software Foundation. (2025). Apache Spark Documentation. https
/spark.apache.org/docs

//:Apache Software Foundation. (2025). Apache Kafka Documentation. https
/kafka.apache.org/documentation

.Apache Software Foundation. (2025). Apache Airflow Documentation
/https://airflow.apache.org/docs

/dbt Labs. (2025). dbt Documentation. https://docs.getdbt.com

//:Snowflake Inc. (2025). Snowflake Documentation. https
/docs.snowflake.com

English Translation Section

Chapter 1: What is Data Engineering?

Definition of Data Engineering

Data Engineering is a branch of software engineering that focuses on designing, building, and maintaining the systems and infrastructure needed to collect, move, store, and process data. A data engineer ensures that data flows smoothly from its various sources to required destinations, whether they are data warehouses, analytics systems, or machine learning applications.

Think of a data engineer as a civil engineer who builds the roads, bridges, and tunnels that connect cities. Just as a civil engineer designs transportation infrastructure, a data engineer designs the infrastructure for data transport and storage.

Data Engineering vs. Data Science

Data Engineering and Data Science are often confused, but they are distinct and complementary disciplines:

Criteria	Data Engineering	Data Science
Primary Focus	Building and maintaining data infrastructure	Analyzing data and extracting patterns
Key Skills	SQL, Python, Spark, ETL, Cloud	Statistics, ML, Data Visualization
Outputs	Data pipelines, warehouses, APIs	Predictive models, reports, insights
Tools	Airflow, Kafka, Spark, dbt	Jupyter, TensorFlow, scikit-learn, R
Analogy	Building the water network	Analyzing water quality

Importance of Data Engineering in the Digital Age

The importance of data engineering continues to grow for several key reasons: the explosion in data volume (global data doubles approximately every two years), the shift toward data-driven decision making, the rise of AI and machine learning (which require robust data infrastructure), regulatory compliance requirements (GDPR, CCPA), and the competitive advantage that strong data infrastructure provides.

Chapter 2: Essential Skills for Data Engineers

Programming Languages

Python

Python is the most widely used language in data engineering due to its ease of learning, rich libraries (Pandas, PySpark, SQLAlchemy), and extensive community support. Data engineers use it for writing ETL scripts, working with APIs, and data processing.

SQL

SQL (Structured Query Language) is the fundamental language for working with databases. No data engineer can succeed without advanced SQL proficiency. It is used for queries, table creation, performance optimization, and creating views.

Java and Scala

Java and Scala are widely used in the Apache ecosystem. Apache Spark is written in Scala, and Apache Kafka is written in Java. Advanced data engineers need to understand at least one of these languages for working with distributed systems.

Bash / Shell Scripting

An essential skill for automating daily tasks such as file transfers, scheduling cron jobs, and monitoring systems.

Databases: Relational and Non-Relational

Data engineers must understand the differences between database types and choose the most appropriate one for each use case. Relational databases (PostgreSQL, MySQL) are best for structured data and financial transactions. Document databases (MongoDB) suit semi-structured, variable content. Columnar databases (Cassandra, HBase) excel at time-series and IoT data. Key-value stores (Redis, DynamoDB) are ideal for caching and user sessions.

Graph databases (Neo4j) serve social networks and recommendation systems.

Chapter 3: Databases and Storage Systems

Relational Databases

PostgreSQL

PostgreSQL is one of the most powerful open-source relational databases. It features support for advanced data types (JSON, Array, Geospatial), high performance, and powerful extensions like PostGIS for geographic data and TimescaleDB for time series.

MySQL

MySQL is the most widely deployed database in web applications, known for fast reads and easy setup. The InnoDB engine supports transactions and foreign keys.

NoSQL Databases

MongoDB stores data in BSON format (Binary JSON), suitable for applications requiring flexibility. Apache Cassandra is a distributed columnar database designed for massive data volumes across multiple servers. Redis is an in-memory data store for ultra-fast caching and message brokering.

Data Warehouses

A data warehouse is a centralized storage system designed for analytics and reporting. Key characteristics include being subject-oriented, integrated, non-volatile, and time-variant. Major platforms include Amazon Redshift, Google BigQuery, Snowflake, Azure Synapse, and Databricks.

Data Lakes

A data lake is a centralized storage repository that holds data in its raw format until needed. Unlike data warehouses, data lakes do not require organizing data before storage. The Lakehouse concept has

emerged recently, combining the benefits of both data lakes and data warehouses.

Chapter 4: Advanced SQL for Data Engineers

Complex Queries and Window Functions

Window functions are among the most important skills for data engineers. They allow performing calculations on a set of rows related to the current row without reducing the number of rows. Common window functions include ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM OVER, AVG OVER, and NTILE.

Performance Optimization and Indexes

Query performance optimization is a critical skill. Data engineers must understand execution plans and choose appropriate indexes. Types include B-Tree (default, for comparisons), Hash (equality

only), GIN (arrays and JSONB), GiST (geographic and full-text), and BRIN (large naturally-ordered tables).

CTEs and Advanced Subqueries

Common Table Expressions (CTEs) make complex queries more readable and maintainable. Recursive CTEs are particularly useful for building hierarchical structures like organizational trees.

Chapter 5: The Apache Ecosystem

Apache Hadoop

Apache Hadoop is an open-source framework for distributed processing of large datasets. It consists of four main components: HDFS (distributed file system), YARN (resource manager), MapReduce (processing model), and Hadoop Common (shared utilities).

Apache Spark

Apache Spark is a unified distributed computing engine supporting batch processing, stream processing, machine learning, and graph processing. It is the most important tool in a modern data engineer's toolkit. Components include Spark Core, Spark SQL, Spark Streaming/Structured Streaming, MLlib, and GraphX.

Apache Hive

Apache Hive is a data warehouse system built on Hadoop that provides a SQL interface (HiveQL) for querying data stored in HDFS. Key features include Schema on Read, Metastore for centralized metadata, Partitioning for query optimization, and Bucketing for improved join performance.

Apache Kafka

Apache Kafka is a distributed event streaming platform used for building real-time data pipelines. It can handle millions of messages per second. Core concepts include Topics (data channels), Partitions (for load distribution), Producers (publishers), Consumers (readers), Consumer Groups, and Brokers (servers).

Apache Airflow

Apache Airflow is a platform for managing, scheduling, and monitoring workflows. It uses the concept of DAG (Directed Acyclic Graph) to define task sequences and their dependencies.

Chapter 6: Data Pipelines (ETL/ELT)

ETL vs. ELT

ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) are two fundamental patterns for moving data from sources to destinations. Traditional ETL transforms data on an intermediate server before loading, while modern ELT loads raw data first then transforms within the warehouse using SQL.

Popular ETL/ELT Tools

dbt (Data Build Tool) is a modern transformation tool that works inside the data warehouse using SQL with testing, documentation, and version control support. Apache NiFi provides a visual data integration platform. Fivetran and Airbyte offer automated extraction and loading from hundreds of sources.

Designing Effective Pipelines

Key principles include idempotency (same operation produces same result), monitoring (tracking every stage), scalability (handling growing data volumes), documentation (documenting every stage and transformation rule), error handling (retry mechanisms and dead letter queues), and data testing (validating data quality at each stage).

Chapter 7: Cloud Computing for Data Engineers

Amazon Web Services (AWS)

AWS is the most widely used cloud platform, offering comprehensive data services: S3 for object storage, Redshift for petabyte-scale data warehousing, Glue for serverless ETL, EMR for running Spark/Hadoop/Hive, and Kinesis for real-time data streaming.

Google Cloud Platform (GCP)

GCP offers BigQuery (serverless data warehouse processing petabytes in seconds), Cloud Dataflow (unified batch and stream processing built on Apache Beam), and Cloud Pub/Sub (real-time messaging for event-driven systems).

Microsoft Azure

Azure provides Synapse Analytics (comprehensive analytics combining warehousing and big data), Data Factory (cloud ETL/ELT with visual interface), and Azure Databricks (Apache Spark-based analytics optimized for Azure).

Chapter 8: Real-time Data Processing

Streaming vs. Batch

Batch processing handles large volumes of collected data at scheduled intervals, suitable for historical analytics. Stream processing handles data immediately upon arrival with minimal latency, suitable for real-time alerts, fraud detection, and instant recommendations.

Lambda and Kappa Architectures

Lambda architecture combines batch and speed layers for comprehensive views but suffers from duplicated logic. Kappa architecture uses only the streaming layer, treating all data as a continuous stream — simpler in design but requiring powerful streaming systems.

Apache Kafka Streams and Apache Flink

Kafka Streams is a Java library for stream processing directly from Kafka, notable for its simplicity. Apache Flink is a powerful stream processing engine supporting both real-time and batch processing with excellent stateful processing and windowing capabilities.

Chapter 9: Data Quality and Governance

Data Quality Foundations

Data quality is the backbone of any successful data system. The six dimensions of data quality are: Accuracy, Completeness, Consistency, Timeliness, Validity, and Uniqueness. Key tools include Great Expectations (Python testing framework), dbt Tests, and Soda.

Metadata Management

Metadata is "data about data," including information about structure, source, quality, and relationships. Tools include Apache Atlas, DataHub (from LinkedIn), Amundsen (from Lyft), and dbt Docs. A data catalog serves as a central index of all data assets in an organization.

Compliance and Privacy

GDPR (EU regulation) requires explicit consent, right to deletion, 72-hour breach notification, and a Data Protection Officer. CCPA provides similar protections for California consumers. Key protection techniques include encryption, anonymization, masking, and role-based access control (RBAC).

Chapter 10: Career Path for Data Engineers

Where Do Data Engineers Work?

Data engineers work across all industries: major tech companies (Google, Amazon, Meta), financial services (banks, insurance, investment firms), e-commerce (Amazon, Alibaba, Shopify), healthcare (hospitals, pharmaceutical companies), transportation and logistics (Uber, Lyft, FedEx), consulting firms (Deloitte, McKinsey, Accenture), startups, and freelancing platforms (Toptal, Upwork).

Salaries by Experience and Region

In the United States, junior data engineers (0-2 years) earn $80,000-$110,000, mid-level (2-5 years) earn $110,000-$150,000, senior (5-8 years) earn $150,000-$200,000, and staff/

principal level (8+ years) can earn $200,000-$300,000+. European salaries range from €40,000-€130,000 depending on level and country. Middle Eastern salaries range from $40,000-$160,000, often tax-free in the Gulf region.

Qualifications and Certifications

Academic paths include bachelor's degrees in Computer Science, Software Engineering, Information Systems, Mathematics, or Statistics. Key certifications include AWS Data Analytics Specialty, Google Professional Data Engineer, Azure Data Engineer Associate, Databricks certifications, and Snowflake SnowPro Core.

Job Interviews

Data engineering interviews typically include SQL questions (complex queries, window functions, optimization), system design (designing a complete data pipeline), programming (Python/Java focused on data processing), concepts (storage patterns, distributed computing principles), and practical experience discussions.

Chapter 11: Practical Projects

Project 1: Building a Complete Data Pipeline

This project involves building an end-to-end data pipeline that collects weather data from an external API, processes it, and stores it in a data warehouse for analysis. Components include the source (OpenWeatherMap API), extraction (Python script scheduled with Airflow), raw storage (S3 in JSON format), transformation (Spark or dbt), the warehouse (PostgreSQL or Redshift), and visualization (Grafana or Metabase).

Project 2: Building a Data Warehouse

This project demonstrates building a data warehouse using the Star Schema model for sales data analysis, including dimension tables (date, product, customer) and a fact table (sales) with proper relationships and optimized query patterns.

Chapter 12: The Future of Data Engineering

AI and Data Engineering

Artificial intelligence is transforming data engineering through automated pipeline construction (Auto-ETL), intelligent data quality monitoring using ML for anomaly detection, natural language querying interfaces, and MLOps integration.

Future Trends

Key trends include Data Mesh (decentralized data ownership), Real-time Everything (streaming as the default), Data Contracts (formal agreements between producers and consumers), Declarative Data Pipelines, Lakehouse Architecture, and Edge Computing (processing data near its source).

Conclusion

Data engineering is not just a collection of tools and technologies — it is a philosophy for handling data and transforming it into valuable assets. The successful engineer understands fundamental principles and can apply them regardless of the specific tool being used. We encourage readers to start practical projects immediately after reading this book, as true learning comes from hands-on application and facing real challenges.

And Allah knows best

نبذة عن المؤلف

د. فضل محمد الأكوع

باحث ومؤلف يمني متخصّص في العلوم والتقنية. يجمع بين الخلفية الأكاديمية العلمية والشغف بتبسيط المعرفة ونشرها باللغة العربية. يؤمن بأنّ المعرفة التقنية يجب أن تكون متاحة للجميع بلغتهم الأمّ.

ألّف عدّة كتب في مجالات متنوعة تجمع بين العلوم الحديثة والثقافة الإسلامية، ساعياً إلى بناء جسور بين المعرفة العلمية المعاصرة والتراث الإسلامي. يحرص في مؤلفاته على تقديم المحتوى بشكل ثنائي اللغة (عربي - إنجليزي) لتوسيع دائرة الاستفادة.

Dr. Fadhl Mohammed Al-Akwa'a

A Yemeni researcher and author specializing in science and technology. He combines an academic scientific background with a passion for simplifying knowledge and making it accessible in Arabic. He believes that technical knowledge should be available to everyone in their native language.

He has authored several books across diverse fields that bridge modern sciences and Islamic culture, striving to build connections between contemporary scientific knowledge and Islamic heritage. His publications are presented bilingually (Arabic-English) to maximize their reach and benefit.

مؤلفات المؤلف

قائمة المؤلفات المنشورة

#	عنوان الكتاب	السنة
1	نهاية الكون في القرآن الكريم — تحليل علمي شامل	2026
2	هندسة البيانات — الدليل الشامل لاحتراف هندسة البيانات	2026

Published Works

#	Book Title	Year
1	The End of the Universe in the Holy Quran — A Comprehensive Scientific Analysis	2026
2	Data Engineering — The Complete Guide to Mastering Data Engineering	2026

عن الكتاب

يقدم هذا الكتاب دليلا شاملا في هندسة البيانات
من المفاهيم الاساسية الى التقنيات المتقدمة
يغطي ادوات مثل اباتشي سبارك وهايف وكافكا
ويرشد القارئ لاحتراف بناء انظمة البيانات

عن المؤلف

د. فضل محمد الاكوع
باحث ومؤلف متخصص في العلوم والتقنية
له عدة مؤلفات في مجالات متنوعة

جميع الحقوق محفوظة للمؤلف © 2026